Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

Лабораторная работа №5

«Разработка простого бота для Telegram с использованием языка Python»

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-32Б _____ (Подругин Г.Р.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

Цель и задание

Цель работы — изучить разработку ботов для Telegram на языке Python.

Разработать простого Telegram-бота, использующего кнопки.

Реализован учебный ежедневник для группы ИУ5-32Б. Бот показывает занятия на сегодня, завтра и неделю, принимает новое расписание и отправляет занятия ежедневно в выбранное время.

Описание реализации

Использованы Python 3.10 или новее, python-telegram-bot 22.8, python-dotenv 1.2.1 и SQLite. Модули schedule.py, storage.py и app.py содержат общую логику двух лабораторных. У каждого личного чата отдельные расписание, время рассылки и отметка последней отправки.

Меню создаётся ReplyKeyboardMarkup. Кнопки «Сегодня», «Завтра» и «Неделя» вызывают соответствующие обработчики; команды /today и текст «расписание» также возвращают занятия. После /start готовое расписание подключается автоматически, ежедневная отправка включается на 08:00.

Учебные недели чередуются относительно известного понедельника числителя — 26 октября 2026 года. Неделя 19–25 октября является знаменателем. В исходном расписании 25 записей: 18 занятий на неделе ЗН и 16 на неделе ЧС. Пятница — день самостоятельной работы, воскресенье свободно.

Для ответа по запросу используется дата отправки сообщения в Europe/Moscow; задержка обработки через полночь не меняет день ответа. JobQueue каждые 30 секунд проверяет время ежедневной отправки и вычисляет расписание на текущую дату. SQLite сохраняет подтверждённые данные и дату отправки после перезапуска.

Простой бот различает нажатия кнопок и данные расписания без конечного автомата. Текст или TXT проверяется и сохраняется сразу. При ошибке старые данные остаются без изменений. Время меняется через /notify ЧЧ:ММ, отправка выключается командой /off.

Порядок запуска

Из корня репозитория создать виртуальное окружение и установить requirements.txt. Скрипт запуска запрашивает токен скрытым вводом и сохраняет только в локальном .env, исключённом из Git.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python telegram_labs/launch.py --lab 5
```

В Telegram открыть @velisbro_study_diary_bot и отправить /start. Одновременно с одним токеном запускается только одна версия. Для ежедневной отправки нужны интернет и постоянно работающая программа; спящий компьютер сообщения не отправляет.

Результаты выполнения

Экранные формы получены в реальном Telegram. На рисунках показаны текстовый ответ с датой и типом недели, а также кнопка меню. Показан общий интерфейс модуля app.py, который используют обе версии бота.

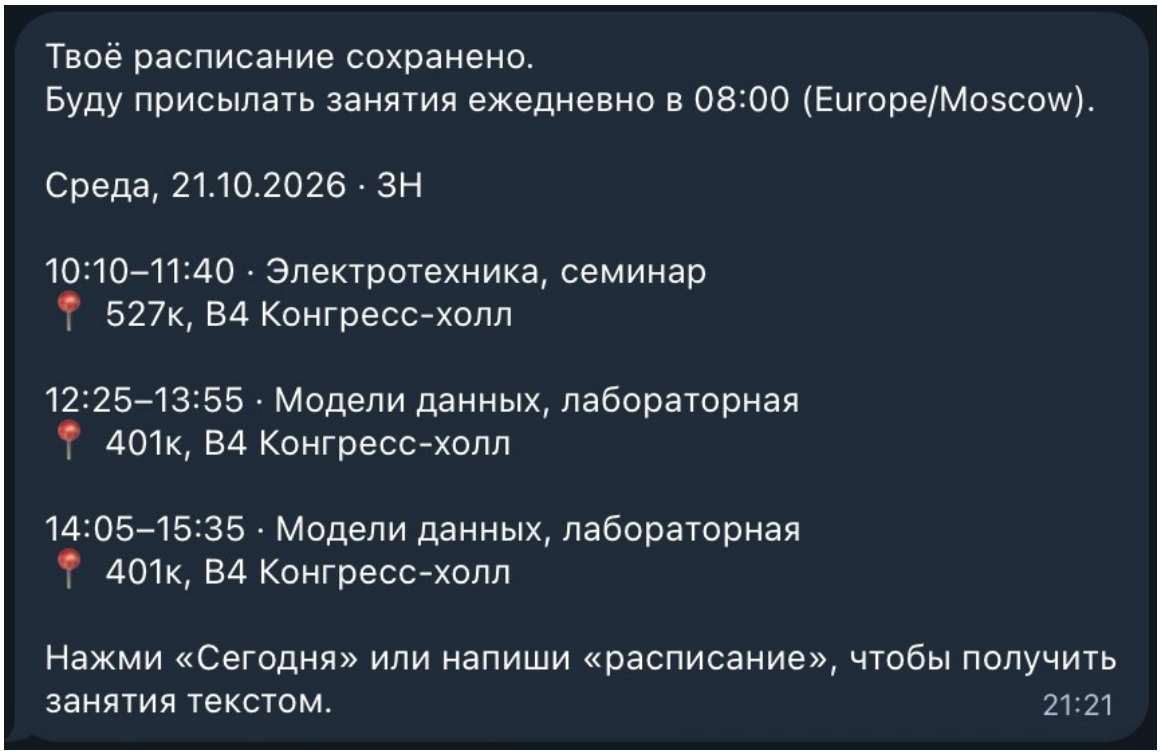


Рисунок 1 — Ответ на запрос текущего дня

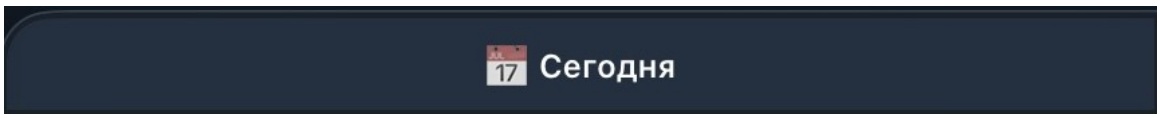


Рисунок 2 — Кнопка Сегодня в клавиатуре бота

Проверка и выводы

Для ЛР 5 выполнено 12 проверок pytest; общий набор репозитория содержит 66 тестов, все завершились успешно. Журналы находятся в results/tests.txt и validation/tests.txt. Локальный транспорт подменяет Bot API, сохраняя настоящий диспетчер python-telegram-bot.

Проверены импорт, ЧС/ЗН, сохранение, изоляция пользователей, отмена, время 8:00, повтор ответа после временной ошибки сети и запрос, задержанный через полночь. Реальная работа меню дополнительно проверена в Telegram и подтверждена экранными формами.

Реализован ежедневник с кнопками и текстовой выдачей расписания. Кнопки вызывают обработчики запросов, а загрузка проверяет данные перед сохранением.

Пропущенные при простое дни не рассылаются, после запуска возможна отправка за текущий день. Каникулы и переносы занятий требуют обновления расписания. При аварии после отправки, но до записи отметки в SQLite возможен повтор сообщения.

Источники

https://github.com/ugapanyuk/BKIT_2021/wiki/lab_bot1

https://github.com/ugapanyuk/courses_content/wiki/COURSE_PCPL_MAIN

https://docs.python-telegram-bot.org/en/v22.8/

https://core.telegram.org/bots/api

Далее приведены полные исходные тексты основного и общих модулей, включая скрипт запуска. Эмодзи в печатном листинге представлены эквивалентными Unicode escape последовательностями.

Текст программы

```
# Файл lab_05/main.py

"""ЛР 5: простой Telegram-бот с кнопками и загрузкой расписания."""
from pathlib import Path
import sys

if __package__ in (None, ""):
    sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from telegram import Update
from telegram.ext import CommandHandler, ContextTypes, MessageHandler, filters

from telegram_labs.app import (
    FORMAT, SCHEDULE_INPUT, UPLOAD, add_common_handlers, create_base, read_input,
    reply, route_buttons, run, store,
)
from telegram_labs.schedule import parse_schedule


async def upload(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await reply(update, FORMAT + "\n\nПосле загрузки рассылка включится на 07:00.
    Изменить: /notify 08:00")


async def receive(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if await route_buttons(update, context):
        return
    if update.effective_message.text == UPLOAD:
        await upload(update, context)
        return
    try:
        schedule = parse_schedule(await read_input(update))
    except ValueError as error:
        await reply(update, str(error) + "\n\nФормат: /help")
        return
    old = store(context).get(update.effective_chat.id)
    reminder = old["notify_time"] if old else "07:00"
    store(context).save(update.effective_chat.id, schedule, reminder)
    await reply(update, f"Сохранено занятий: {len(schedule.lessons)}.\n"
                f"Ежедневная отправка: {reminder}
    ({context.bot_data['timezone']}).\n"
                "Нажми «Сегодня», «Завтра» или «Неделя».")


async def cancel(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await reply(update, "Можно пользоваться меню. Сохранённое расписание не
    менялось.")


def build_application(token=None, db_path=None, request=None):
    application = create_base(5, token, db_path, request)
    add_common_handlers(application)
    application.add_handler(CommandHandler("upload", upload,
    filters=filters.ChatType.PRIVATE))
    application.add_handler(CommandHandler("cancel", cancel,
    filters=filters.ChatType.PRIVATE))
    application.add_handler(MessageHandler(SCHEDULE_INPUT, receive))
    return application
```

Текст программы продолжение

```
if __name__ == "__main__":
    run(build_application)

# Файл telegram_labs/schedule.py

"""Разбор текстового расписания и чередование числителя/знаменателя."""
from dataclasses import asdict, dataclass
from datetime import date, timedelta
import json
import re

DAYS = ("Пн", "Вт", "Ср", "Чт", "Пт", "Сб", "Вс")
DAY_NAMES = ("Понедельник", "Вторник", "Среда", "Четверг", "Пятница", "Суббота",
    "Воскресенье")
WEEKS = {"каждую": "all", "чс": "ЧС", "зн": "ЗН"}


def parse_time(value: str) -> str:
    """Проверить время, привести 7:00 к 07:00."""
    if not re.fullmatch(r"\d{1,2}:\d{2}", value.strip()):
        raise ValueError("Время должно быть в формате ЧЧ:ММ, например 07:00.")
    hour, minute = map(int, value.split(":"))
    if not 0 <= hour <= 23 or not 0 <= minute <= 59:
        raise ValueError("Часы: 00–23, минуты: 00–59.")
    return f"{hour:02}:{minute:02}"


@dataclass(frozen=True)
class Lesson:
    day: int
    start: str
    end: str
    subject: str
    room: str
    week: str


@dataclass(frozen=True)
class Schedule:
    lessons: tuple[Lesson, ...]
    numerator_monday: date | None = None
    notes: tuple[tuple[int, str], ...] = ()

    def week_on(self, day: date) -> str:
        if self.numerator_monday is None:
            return "каждую неделю"
        # Целочисленное деление работает и для дат до опорного понедельника.
        weeks = (day - self.numerator_monday).days // 7
        return "ЧС" if weeks % 2 == 0 else "ЗН"

    def on(self, day: date) -> list[Lesson]:
        return sorted((lesson for lesson in self.lessons
                       if lesson.day == day.weekday()
                       and lesson.week in ("all", self.week_on(day))), key=lambda x:
     x.start)

    def render_day(self, day: date) -> str:
```

Текст программы продолжение

```
        lines = [f"{DAY_NAMES[day.weekday()]}, {day:%d.%m.%Y} ·
    {self.week_on(day)}"]
        lessons = self.on(day)
        for lesson in lessons:
            lines.append(f"\n{lesson.start}–{lesson.end} ·
    {lesson.subject}\n\U0001f4cd {lesson.room}")
        if not lessons:
            lines.append("\nЗанятий по расписанию нет.")
        for weekday, note in self.notes:
            if weekday == day.weekday():
                lines.append(f"\n\U0001f4dd {note}")
        return "\n".join(lines)

    def render_week(self, day: date) -> str:
        monday = day - timedelta(days=day.weekday())
        return "\n\n".join(self.render_day(monday + timedelta(days=i)) for i in
    range(7))

    def dumps(self) -> str:
        return json.dumps({"lessons": [asdict(x) for x in self.lessons],
                           "numerator_monday": self.numerator_monday.isoformat() if
    self.numerator_monday else None,
                           "notes": self.notes}, ensure_ascii=False)

    @classmethod
    def loads(cls, raw: str) -> "Schedule":
        data = json.loads(raw)
        return cls(tuple(Lesson(**x) for x in data["lessons"]),
                   date.fromisoformat(data["numerator_monday"]) if
    data["numerator_monday"] else None,
                   tuple(tuple(x) for x in data.get("notes", [])))


def parse_schedule(text: str) -> Schedule:
    """Формат: Пн | 08:30-10:00 | Предмет | Аудитория | ЧС/ЗН/каждую."""
    if len(text) > 40000:
        raise ValueError("Расписание слишком большое: максимум 40 000 символов.")
    lessons, notes = [], []
    anchor = None
    for number, raw in enumerate(text.lstrip("\ufeff").splitlines(), 1):
        raw = raw.strip()
        if not raw or raw.startswith("#"):
            continue
        try:
            if raw.lower().startswith("неделя чс:"):
                if anchor is not None:
                    raise ValueError("Укажи только одну опорную неделю ЧС.")
                anchor = date.fromisoformat(raw.split(":", 1)[1].strip())
                if anchor.weekday() != 0:
                    raise ValueError("Дата недели ЧС должна приходиться на
    понедельник.")
                continue
            parts = [x.strip() for x in raw.split("|")]
            if len(parts) == 3 and parts[1].lower() == "заметка":
                weekday = DAYS.index(parts[0].capitalize())
                if not parts[2] or len(parts[2]) > 300:
                    raise ValueError("Заметка должна содержать от 1 до 300
    символов.")
                notes.append((weekday, parts[2]))
```

Текст программы продолжение

```
                continue
            if len(parts) != 5:
                raise ValueError("Нужно 5 полей: день | время | предмет | аудитория
    | неделя.")
            weekday = DAYS.index(parts[0].capitalize())
            times = re.split(r"\s*[-–—]\s*", parts[1])
            if len(times) != 2:
                raise ValueError("Интервал времени: 08:30-10:00.")
            start, end = map(parse_time, times)
            if start >= end:
                raise ValueError("Начало занятия должно быть раньше окончания.")
            if not parts[2] or len(parts[2]) > 160 or not parts[3] or len(parts[3])
    > 100:
                raise ValueError("Укажи предмет (до 160 символов) и место (до 100
    символов).")
            if parts[4].lower() not in WEEKS:
                raise ValueError("Неделя: ЧС, ЗН или каждую.")
            lesson = Lesson(weekday, start, end, parts[2], parts[3],
    WEEKS[parts[4].lower()])
            for old in lessons:
                same_week = old.week == lesson.week or "all" in (old.week,
    lesson.week)
                if old.day == lesson.day and same_week and old.start < lesson.end
    and lesson.start < old.end:
                    raise ValueError("Занятия пересекаются по времени; проверь
    строки расписания.")
            lessons.append(lesson)
        except ValueError as error:
            raise ValueError(f"Строка {number}: {error}") from None
        if len(lessons) > 100:
            raise ValueError("Максимум 100 занятий.")
    if not lessons:
        raise ValueError("Не найдено занятий. Пришли расписание в указанном
    формате.")
    if anchor is None and any(x.week != "all" for x in lessons):
        raise ValueError("Для ЧС/ЗН добавь строку «Неделя ЧС: ГГГГ-ММ-ДД» с
    понедельником числителя.")
    return Schedule(tuple(lessons), anchor, tuple(notes))

# Файл telegram_labs/storage.py

"""SQLite: отдельное расписание, время рассылки и отметка отправки для каждого
    чата."""
from datetime import datetime
from contextlib import contextmanager
from pathlib import Path
import sqlite3

from .schedule import Schedule, parse_time


class Store:
    def __init__(self, path: str | Path):
        path = Path(path)
        path.parent.mkdir(parents=True, exist_ok=True)
        self.path = path
        with self.connect() as db:
            db.execute("""CREATE TABLE IF NOT EXISTS diaries (
                chat_id INTEGER PRIMARY KEY, schedule TEXT NOT NULL,
```

Текст программы продолжение

```
                notify_time TEXT NOT NULL, enabled INTEGER NOT NULL DEFAULT 1,
                last_sent TEXT)""")

    @contextmanager
    def connect(self):
        db = sqlite3.connect(self.path)
        db.row_factory = sqlite3.Row
        try:
            with db:
                yield db
        finally:
            db.close()

    def get(self, chat_id: int):
        with self.connect() as db:
            row = db.execute("SELECT * FROM diaries WHERE chat_id=?",
    (chat_id,)).fetchone()
            return dict(row) if row else None

    def save(self, chat_id: int, schedule: Schedule, notify_time: str = "07:00"):
        notify_time = parse_time(notify_time)
        with self.connect() as db:
            db.execute("""INSERT INTO diaries (chat_id, schedule, notify_time)
    VALUES (?, ?, ?)
                ON CONFLICT(chat_id) DO UPDATE SET schedule=excluded.schedule,
                notify_time=excluded.notify_time, enabled=1""",
                       (chat_id, schedule.dumps(), notify_time))

    def set_reminder(self, chat_id: int, value: str):
        value = parse_time(value)
        with self.connect() as db:
            db.execute("UPDATE diaries SET notify_time=?, enabled=1 WHERE
    chat_id=?", (value, chat_id))

    def disable(self, chat_id: int):
        with self.connect() as db:
            db.execute("UPDATE diaries SET enabled=0 WHERE chat_id=?", (chat_id,))

    def delete(self, chat_id: int):
        with self.connect() as db:
            db.execute("DELETE FROM diaries WHERE chat_id=?", (chat_id,))

    def due(self, now: datetime):
        with self.connect() as db:
            return [dict(x) for x in db.execute("""SELECT * FROM diaries WHERE
    enabled=1
                AND notify_time <= ? AND (last_sent IS NULL OR last_sent != ?)""",
                (now.strftime("%H:%M"), now.date().isoformat()))]

    def mark_sent(self, chat_id: int, now: datetime):
        with self.connect() as db:
            db.execute("UPDATE diaries SET last_sent=? WHERE chat_id=?",
    (now.date().isoformat(), chat_id))

# Файл telegram_labs/app.py

"""Общие команды, кнопки, загрузка TXT и ежедневная рассылка."""
from datetime import datetime, timedelta
import asyncio
```

Текст программы продолжение

```
from io import BytesIO
import logging
import os
from pathlib import Path
from zoneinfo import ZoneInfo

from dotenv import load_dotenv
from telegram import BotCommand, ReplyKeyboardMarkup, Update
from telegram.error import Conflict, Forbidden, InvalidToken, NetworkError,
    TelegramError
from telegram.ext import Application, CommandHandler, ContextTypes, MessageHandler,
    filters

from .schedule import Schedule, parse_schedule, parse_time
from .storage import Store

ROOT = Path(__file__).resolve().parents[1]
TODAY, TOMORROW, WEEK = "\U0001f4c5 Сегодня", "\u27a1\ufe0f Завтра", "\U0001f5d3
    Неделя"
UPLOAD, SETTINGS, TEMPLATE = "\U0001f4e5 Загрузить расписание", "\u2699\ufe0f
    Уведомления", "\U0001f4c4 Файл расписания"
MENU = ReplyKeyboardMarkup([[TODAY, TOMORROW, WEEK], [UPLOAD], [SETTINGS,
    TEMPLATE]], resize_keyboard=True)
PRIVATE_TEXT = filters.ChatType.PRIVATE & filters.TEXT & ~filters.COMMAND
SCHEDULE_INPUT = filters.ChatType.PRIVATE & (filters.TEXT | filters.Document.ALL) &
    ~filters.COMMAND
FORMAT = (
    "Пришли расписание одним сообщением или файлом .txt в UTF-8.\n"
    "Каждое занятие — отдельная строка:\n"
    "Пн | 08:30-10:00 | Математика, лекция | 501 | каждую\n\n"
    "Для чередующихся недель используй ЧС или ЗН вместо «каждую» и добавь,
    например:\n"
    "Неделя ЧС: 2026-10-26\n"
    "Это дата понедельника известной недели ЧС.\n"
    "Заметка на день: Пт | заметка | День самостоятельной работы\n\n"
    "Кнопка «Файл расписания» или /template выдаёт подготовленное расписание ИУ5-32Б
     по твоим скриншотам."
)
LOG = logging.getLogger(__name__)


def chunks(text: str):
    # 1800 Unicode-символов не превышают 3600 UTF-16 единиц даже с emoji.
    while text:
        split = text.rfind("\n", 0, 1800) if len(text) > 1800 else len(text)
        if split <= 0:
            split = min(1800, len(text))
        yield text[:split]
        text = text[split:].lstrip("\n")


async def reply(update: Update, text: str, markup=MENU):
    for part in chunks(text):
        for attempt in range(2):
            try:
                await update.effective_message.reply_text(part, reply_markup=markup)
                break
            except NetworkError:
                if attempt:
```

Текст программы продолжение

```
                    raise
                LOG.warning("Ответ отложен из-за сетевой ошибки; повторная попытка")
                await asyncio.sleep(0.5)


async def read_input(update: Update) -> str:
    message = update.effective_message
    if message.document:
        document = message.document
        if not (document.file_name or "").lower().endswith(".txt"):
            raise ValueError("Пришли текст или файл .txt. Фото и PDF автоматически
    не распознаются.")
        if document.file_size is None or document.file_size > 100_000:
            raise ValueError("Максимальный размер файла — 100 КБ.")
        file = await document.get_file()
        content = await file.download_as_bytearray()
        if len(content) > 100_000:
            raise ValueError("Максимальный размер файла — 100 КБ.")
        try:
            return content.decode("utf-8-sig")
        except UnicodeDecodeError:
            raise ValueError("Сохрани файл в кодировке UTF-8 и отправь ещё раз.")
    from None
    return message.text or ""


def store(context: ContextTypes.DEFAULT_TYPE) -> Store:
    return context.bot_data["store"]


def now(context: ContextTypes.DEFAULT_TYPE) -> datetime:
    return datetime.now(context.bot_data["timezone"])


def request_day(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Дата отправки запроса пользователем по Москве, даже при задержке
    обработки."""
    return
    update.effective_message.date.astimezone(context.bot_data["timezone"]).date()


def get_diary(update: Update, context: ContextTypes.DEFAULT_TYPE):
    database = store(context)
    chat_id = update.effective_chat.id
    row = database.get(chat_id)
    created = row is None
    if created:
        schedule = parse_schedule((ROOT /
    "telegram_labs/data/iu5_32b.txt").read_text(encoding="utf-8"))
        database.save(chat_id, schedule, "08:00")
        row = database.get(chat_id)
    return row, created


def mark_initial_delivery(update: Update, context: ContextTypes.DEFAULT_TYPE, row,
    created):
    current = now(context)
    if (created and request_day(update, context) == current.date()
            and current.strftime("%H:%M") >= row["notify_time"]):
```

Текст программы продолжение

```
        # Первый ответ уже содержит сегодняшний день; повтор от таймера не нужен.
        store(context).mark_sent(update.effective_chat.id, current)


def clear_draft(context: ContextTypes.DEFAULT_TYPE):
    context.user_data.pop("draft", None)
    context.user_data.pop("draft_time", None)


async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    clear_draft(context)
    row, created = get_diary(update, context)
    status = (f"Буду присылать занятия ежедневно в {row['notify_time']}
    ({context.bot_data['timezone']})."
              if row["enabled"] else "Ежедневная отправка выключена. Включить:
    /notify 08:00")
    title = "Твоё расписание ИУ5-32Б подключено." if created else "Твоё расписание
    сохранено."
    schedule = Schedule.loads(row["schedule"])
    await reply(update, f"{title}\n{status}\n\n" +
    schedule.render_day(request_day(update, context))
                + "\n\nНажми «Сегодня» или напиши «расписание», чтобы получить
    занятия текстом.")
    mark_initial_delivery(update, context, row, created)


async def help_command(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await reply(update, FORMAT + "\n\n/today /tomorrow /week — занятия\n"
                "/notify 07:00 — включить рассылку или изменить время\n"
                "/off — выключить рассылку\n/delete — удалить свои данные")


async def template(update: Update, context: ContextTypes.DEFAULT_TYPE):
    data = (ROOT / "telegram_labs/data/iu5_32b.txt").read_bytes()
    await update.effective_message.reply_document(
        document=BytesIO(data), filename="iu5_32b.txt",
        caption="Расписание ИУ5-32Б: проверь аудитории и подгруппу. Для загрузки
    отправь этот файл боту.")


async def show(update: Update, context: ContextTypes.DEFAULT_TYPE, mode: str):
    row, created = get_diary(update, context)
    schedule = Schedule.loads(row["schedule"])
    day = request_day(update, context)
    if mode == "tomorrow":
        day += timedelta(days=1)
    text = schedule.render_week(day) if mode == "week" else schedule.render_day(day)
    await reply(update, text)
    if mode == "today":
        mark_initial_delivery(update, context, row, created)


async def today(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await show(update, context, "today")


async def tomorrow(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await show(update, context, "tomorrow")
```

Текст программы продолжение

```
async def week(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await show(update, context, "week")


async def settings(update: Update, context: ContextTypes.DEFAULT_TYPE):
    row = store(context).get(update.effective_chat.id)
    if row is None:
        await reply(update, "Сначала загрузи расписание.")
        return
    state = f"каждый день в {row['notify_time']}" if row["enabled"] else "выключены"
    await reply(update, f"Уведомления: {state}. Часовой пояс:
    {context.bot_data['timezone']}.\n"
                "Изменить время: /notify 07:00\nВыключить: /off\n"
                "Программа должна работать постоянно. Если включить её после
    выбранного времени, "
                "она пришлёт расписание за текущий день в течение минуты.")


async def notify(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not store(context).get(update.effective_chat.id):
        await reply(update, "Сначала загрузи расписание.")
        return
    try:
        if len(context.args) != 1:
            raise ValueError("Пример команды: /notify 07:00")
        value = parse_time(context.args[0])
    except ValueError as error:
        await reply(update, str(error))
        return
    store(context).set_reminder(update.effective_chat.id, value)
    await reply(update, f"Буду присылать занятия ежедневно в {value}
    ({context.bot_data['timezone']}).")


async def off(update: Update, context: ContextTypes.DEFAULT_TYPE):
    store(context).disable(update.effective_chat.id)
    await reply(update, "Ежедневная отправка выключена. Включить снова: /notify
    07:00")


async def delete(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if context.args != ["confirm"]:
        await reply(update, "Чтобы удалить своё расписание и отключить рассылку,
    отправь /delete confirm")
        return
    store(context).delete(update.effective_chat.id)
    clear_draft(context)
    await reply(update, "Твоё расписание удалено, рассылка отключена.")


async def route_buttons(update: Update, context: ContextTypes.DEFAULT_TYPE) -> bool:
    actions = {TODAY: today, TOMORROW: tomorrow, WEEK: week,
               SETTINGS: settings, TEMPLATE: template,
               "сегодня": today, "расписание": today, "расписание на сегодня":
    today,
               "какие пары сегодня": today, "завтра": tomorrow, "неделя": week}
    text = (update.effective_message.text or "").strip().casefold().rstrip("?!.")
    action = {key.casefold(): value for key, value in actions.items()}.get(text)
```

Текст программы продолжение

```
    if action:
        await action(update, context)
        return True
    return False


async def reminders(context: ContextTypes.DEFAULT_TYPE):
    current = now(context)
    database = store(context)
    for row in database.due(current):
        try:
            text = "\u2600\ufe0f Занятия на сегодня\n\n" +
    Schedule.loads(row["schedule"]).render_day(current.date())
            for part in chunks(text):
                await context.bot.send_message(chat_id=row["chat_id"], text=part)
            # Запись после успешной отправки не допускает обычных повторов после
    перезапуска.
            database.mark_sent(row["chat_id"], current)
        except Forbidden:
            database.disable(row["chat_id"])
        except TelegramError as error:
            # Не логируем URL запроса: в URL Telegram API содержится токен.
            LOG.warning("Рассылка отложена: %s", type(error).__name__)


async def post_init(application: Application):
    await application.bot.set_my_commands([
        BotCommand("start", "Открыть ежедневник"), BotCommand("upload", "Загрузить
    расписание"),
        BotCommand("today", "Занятия сегодня"), BotCommand("tomorrow", "Занятия
    завтра"),
        BotCommand("week", "Текущая неделя"), BotCommand("notify", "Настроить
    ежедневную отправку"),
        BotCommand("off", "Отключить отправку"), BotCommand("template", "Файл
    расписания ИУ5-32Б"),
        BotCommand("help", "Помощь"), BotCommand("cancel", "Отменить ввод")])


async def error_handler(update: object, context: ContextTypes.DEFAULT_TYPE):
    LOG.error("Ошибка обработки: %s", type(context.error).__name__)
    if isinstance(update, Update) and update.effective_message:
        try:
            await reply(update, "Не удалось обработать запрос. Попробуй ещё раз или
    отправь /start.")
        except TelegramError:
            pass


def create_base(lab: int, token: str | None = None, db_path: Path | None = None,
    request=None) -> Application:
    load_dotenv(ROOT / "telegram_labs/.env")
    token = token or os.getenv("BOT_TOKEN", "").strip()
    if not token or token == "PASTE_NEW_TOKEN_HERE":
        raise ValueError("Укажи новый BOT_TOKEN в telegram_labs/.env (пример:
    .env.example).")
    timezone = ZoneInfo(os.getenv("BOT_TIMEZONE", "Europe/Moscow"))
    builder =
    Application.builder().token(token).concurrent_updates(False).post_init(post_init
    )
```

Текст программы продолжение

```
    if request is not None:
        builder = builder.request(request).get_updates_request(request)
    else:
        builder =
    builder.connect_timeout(20).read_timeout(30).write_timeout(30).pool_timeout(20)
    application = builder.build()
    application.bot_data.update(store=Store(db_path or ROOT /
    f"lab_{lab:02}/data/diary.sqlite3"),
                                timezone=timezone, lab=lab)
    application.job_queue.run_repeating(reminders, interval=30, first=1,
                                        job_kwargs={"max_instances": 1, "coalesce":
     True})
    application.add_error_handler(error_handler)
    return application


def add_common_handlers(application: Application, include_start=True):
    commands = {"help": help_command, "today": today, "tomorrow": tomorrow, "week":
    week,
                "notify": notify, "off": off, "delete": delete, "template":
    template}
    if include_start:
        commands["start"] = start
    for name, callback in commands.items():
        application.add_handler(CommandHandler(name, callback,
    filters=filters.ChatType.PRIVATE))


def run(factory):
    logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")
    for name in ("httpx", "httpcore", "telegram", "apscheduler"):
        logging.getLogger(name).setLevel(logging.CRITICAL)
    try:
        application = factory()
    except (ValueError, KeyError) as error:
        raise SystemExit(str(error)) from None
    print("Учебный ежедневник запущен. Остановка: Ctrl+C.")
    try:
        application.run_polling(allowed_updates=["message"],
    drop_pending_updates=False)
    except InvalidToken:
        raise SystemExit("Telegram отклонил токен. Повторный ввод: python
    telegram_labs/launch.py --configure") from None
    except Conflict:
        raise SystemExit("Этот бот уже запущен в другом процессе. Останови второй
    экземпляр и повтори запуск.") from None
    except NetworkError:
        raise SystemExit("Не удалось подключиться к Telegram. Проверь интернет и
    повтори запуск.") from None
    except TelegramError as error:
        raise SystemExit(f"Ошибка Telegram: {type(error).__name__}. Проверь
    настройки и повтори запуск.") from None

# Файл telegram_labs/launch.py

"""Первый запуск без редактирования кода и без показа токена в терминале."""
import argparse
from getpass import getpass
from importlib import metadata
```

Текст программы продолжение

```
import os
from pathlib import Path
import re
import subprocess
import sys

ROOT = Path(__file__).resolve().parents[1]


def main():
    parser = argparse.ArgumentParser(description="Запуск учебного ежедневника")
    parser.add_argument("--lab", type=int, choices=(5, 6), default=6)
    parser.add_argument("--configure", action="store_true", help="Ввести новый
    токен")
    args = parser.parse_args()
    if sys.version_info < (3, 10):
        raise SystemExit("Нужен Python 3.10 или новее.")
    requirements = ROOT / "requirements.txt"
    install = False
    for line in requirements.read_text().splitlines():
        name, version = line.split("==")
        try:
            install |= metadata.version(name.split("[")[0]) != version
        except metadata.PackageNotFoundError:
            install = True
    if install:
        subprocess.run([sys.executable, "-m", "pip", "install", "-r",
    str(requirements)], check=True)
    from dotenv import dotenv_values, set_key

    config = ROOT / "telegram_labs/.env"
    saved = dotenv_values(config).get("BOT_TOKEN", "")
    effective = os.getenv("BOT_TOKEN") or saved
    if args.configure or not effective or effective == "PASTE_NEW_TOKEN_HERE":
        print("Введи новый токен BotFather после /revoke. Ввод скрыт, символы не
    отображаются.")
        token = getpass("Токен: ").strip()
        if not re.fullmatch(r"\d+:[A-Za-z0-9_-]{20,}", token):
            raise SystemExit("Формат токена не распознан. Запусти ещё раз и вставь
    токен целиком.")
        config.touch(mode=0o600, exist_ok=True)
        config.chmod(0o600)
        set_key(config, "BOT_TOKEN", token)
        set_key(config, "BOT_TIMEZONE", "Europe/Moscow")
        # Явный --configure должен заменить и переменную окружения текущего запуска.
        os.environ["BOT_TOKEN"] = token
        print("Настройки сохранены локально. Токен не включается в Git.")
    print(f"Лабораторная №{args.lab}. Открой @velisbro_study_diary_bot и отправь
    /start.")
    completed = subprocess.run([sys.executable, str(ROOT /
    f"lab_{args.lab:02}/main.py")], cwd=ROOT)
    raise SystemExit(completed.returncode)


if __name__ == "__main__":
    try:
        main()
    except (KeyboardInterrupt, EOFError):
        print("\nЗапуск отменён.")
```